arXiv:1909.12641v1 [cs.LG] 27 Sep 2019

Active Federated Learning

Jack Goetz
University of Michigan

Kshitiz Malik
Facebook Assistant

Duc Bui
University of Michigan

Seungwhan Moon
Facebook Assistant

Honglei Liu
Facebook Assistant

Anuj Kumar
Facebook Assistant

Abstract

Federated Learning allows for population level models to be trained without centralizing client data by transmitting the global model to clients, calculating gradients locally, then averaging the gradients. Downloading models and uploading gradients uses the client's bandwidth, so minimizing these transmission costs is important. The data on each client is highly variable, so the benefit of training on different clients may differ dramatically. To exploit this we propose *Active Federated Learning*, where in each round clients are selected not uniformly at random, but with a probability conditioned on the current model and the data on the client to maximize efficiency. We propose a cheap, simple and intuitive sampling scheme which reduces the number of required training iterations by 20-70% while maintaining the same model accuracy, and which mimics well known resampling techniques under certain conditions.

1 Introduction

As machine learning models are deployed in the real world, the assumptions under which they were developed are often shown to be incompatible with user requirements. One assumptions is unrestricted access to the training data, either on a single machine or distributed over many researcher controlled machines. Due to privacy concerns users may not want to transmit data from their personal devices, making such centralized training impossible. Federated Learning enables the training of models on this data, but transmission costs between the server and the client are high, and reducing these costs is important. In this paper we introduce *Active Federated Learning* (AFL) to preferentially train on users which are more beneficial to the model during that training iteration. Motivated by ideas from Active Learning, we propose using a value function which can be evaluated on the user's device and returns a valuation to the server indicating the likely utility of training on that user. The server collects these valuations and converts them to probabilities with which the next cohort of users is selected for training. By using simple a value function related to the loss the user's data suffers under the current model, we can reduce the number of training rounds required for the model to achieve a specified level of accuracy by 20-70%.

2 Related Work

Since its introduction [11, 14], reducing the communication costs of Federated Learning has been an important goal [8, 4]. However as discussed in Li et al. [10] there are few existing techniques which change the method of selecting users. In Hartmann [6] the author suggests stratification based on contextual information about the users, and in Nishio and Yonetani [12] the authors group users based on hardware characteristics. In contrast our work is closer to Active Learning (AL) [13] where the selection policy is dependant on the current state of the model and the data on each user.

Preprint. Under review.

In both paradigms training data must be selected under imperfect information; in AL the covariates are fully known, but the label of candidate data points is unknown, whereas in AFL both labels and covariates are fully known on each client, but only a summary is returned to the server. Additionally, in standard AL individual data points may be selected in an unconstrained manner, whereas in AFL we train on all data points on each selected user, creating predetermined subsets of data.

3 Background and Notation

Assume we have labelled data (x, y) and a model for predicting $y \in \mathcal{Y}$ given $x \in \mathcal{X}$ which we denote by $\hat{y} = f(x; \mathbf{w})$, where $\mathbf{w} \in \mathbb{R}^d$ are our model parameters. These model parameters will be learned by minimizing some loss function $l(x, y; \mathbf{w})$. Assume our training data is distributed over multiple clients (or users) $\mathcal{U} = \{U_1, ..., U_K\}$, where we denote the data of client U_k by $(\mathbf{x}_k, \mathbf{y}_k) \in \mathcal{X}^{n_k} \times \mathcal{Y}^{n_k}$. Our model parameters will be learned during training iterations, so we will let $\mathbf{w}^{(t)}$ denote the value of our parameters at training iteration t. During each training iteration we select a subset of users $\mathcal{S}^{(t)} \subset \mathcal{U}, |\mathcal{S}^{(t)}| = m$ and send $\mathbf{w}^{(t)}$ to each user in the set. Each user then performs some training $\mathcal{T}$ using their local data and produce updated model parameter values $\mathbf{w}_k^{(t+1)} = \mathcal{T}(\mathbf{x}_k, \mathbf{y}_k; \mathbf{w}^{(t)})$. In its most simple form this training could be a single step of gradient descent, though in practice it is often more complicated, such as multiple passes of SGD. These updated model parameter values are then returned to the server and aggregated to produce the next model parameters using Federated ADAM [9]. In traditional Federated Learning the subsets $\mathcal{S}^{(t)}$ are selected uniformly at random and independently at each iteration. Our goal in AFL is to select our subsets $\mathcal{S}^{(t)}$ such that fewer training iterations are required to obtain a good model.

4 Active Federated Learning (AFL)

Inspired by the structure of classical AL methods, we propose the AFL framework which aims to select an optimized subset of users based on a *value* function that reflects how useful the data on that user is during each training round. Formally, we define a function $\mathcal{V} : \mathcal{X}^{n_k} \times \mathcal{Y}^{n_k} \times \mathbb{R}^d \to \mathbb{R}$ which is evaluated on each user. Once evaluated, each user U_k returns a corresponding *valuation* $v_k \in \mathbb{R}$ to the server, which is used to calculate the sampling distribution for the next training iteration. The valuations are a function of $\mathbf{w}^{(t)}$, but since transmitting the model is expensive we only get fresh valuations of users during an iteration in which we train on them, meaning that

$$v_k^{(t+1)} = \begin{cases} \mathcal{V}(\mathbf{x}_k, \mathbf{y}_k; \mathbf{w}^{(t)}) & \text{if } U_k \in S_t \\ v_k^{(t)} & \text{otherwise.} \end{cases}$$

Ideally the computation of the value function should require minimal additional computation, since the computations are done using the clients hardware, and should not reveal too much about the data on each client. Once the server has all valuations it converts them into a sampling distribution.

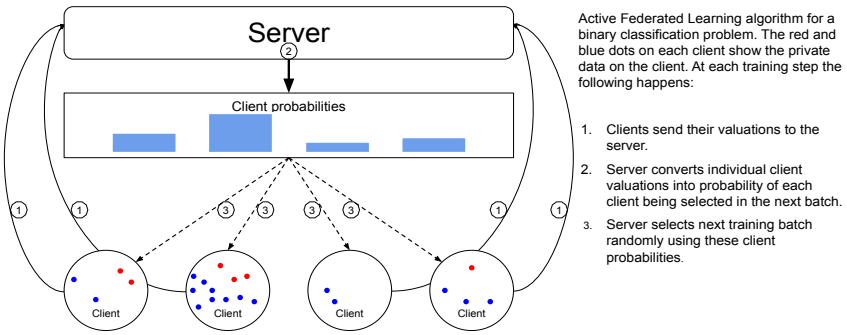


Figure 1: **Active Federated Learning framework** for a binary classification problem.

Algorithm 1: Sampling algorithm

Input: Client Valuations $\{v_1, ..., v_K\}$, tuning parameters $\alpha_1, ..., \alpha_3$, number of clients per round m
Output: Client indices $\{k_1, ..., k_m\}$
Sort users by v_k
For the $\alpha_1 K$ users with smallest v_k, $v_k = -\infty$
for k *from* 1 *to* K **do**
  $p_k \propto e^{\alpha_2 v_k}$
end
Sample $(1 - \alpha_3)m$ users according to their p_k, producing set $\mathcal{S}'$
Sample $\alpha_3 m$ from the remaining users uniformly at random, producing set $\mathcal{S}''$
return $\mathcal{S} = \mathcal{S}' \cup \mathcal{S}''$

4.1 Loss valuation

One very natural value function is to use the loss of the users data $v_k = \frac{1}{\sqrt{n_k}} l(\mathbf{x}_k, \mathbf{y}_k; \mathbf{w})$. It is already calculated during model training and is increasing with how poorly the model performs on the clients data. Additionally it mimics common resampling techniques when the required structure is present in the data. If there is extreme class imbalance and weak separation of the classes, data points of the minority class will have significantly higher loss than majority class data points. Therefore we will prefer users with more minority data, mimicking resampling the minority class data. Similarly if the noise depends on the distance from the classification boundary such as in [3], using the loss replicates margin based resampling techniques. Finally if all data points are equally valuable then users with more data will be given higher valuations. Most importantly these adaptations to the data do not require the practitioner to know the specific structure being exploited. This is particularly important in the Federated setting, where information about the data is limited.

4.2 Differential Privacy

Even summarizing the client data with a single float may reveal too much information. To properly protect users the value function should be reported using a Differentially Private mechanism [5]. The noise introduced to maintain Differential Privacy may mislead the server into selecting suboptimal clients. However there is structure which might be exploited to reduce the corruption while still maintaining privacy. One is that many value functions, such as the loss, are not expected to change dramatically within a small number of training rounds. Thus we may be able to query whether a valuation has changed dramatically before querying the new value, similar to the Sparse Vector technique, to reduce the number of queries. We may also be able to adapt our value function to be more amenable to Differential Privacy. For example the loss value function has unbounded sensitivity and requires clipping to provide Differential Privacy. However returning a count of high loss data points has sensitivity 1 and may be less affected by the privacy providing noise. Adding privacy guarantees is an important challenge in AFL and is the subject of much future work.

5 Experimental Results

We compared AFL to the standard uniform selection on two datasets; one on the Reddit dataset, the other on the Sticker Intent dataset. The Reddit dataset is a publicly available [2] dataset consisting of comments from users on *reddit.com*. The authors were not involved in collecting this dataset. For the Reddit dataset we predicted the binary label 'controversially' based on the comment text, and selected 8K users at random from the November 2017 data set, similar to Bagdasaryan et al. [1] but only excluding users with +100K messages. We removed comments being responded to from the messages, and empty messages. The Reddit dataset has many users who post few comments, but a long tail of power users. The Sticker Intent dataset has randomly selected, anonymized messages from a popular messaging app. The task was binary classification - predict whether a message was replied to using a sticker. Messages in this set were collected, de-identified, and annotated automatically; the messages were not read or labeled by human annotators.

Algorithm 1 for converting the valuations into a sampling distribution has 3 tuning parameters: The α_1 proportion of users with the smallest valuations will have their valuations set to $-\infty$. They

Table 1: Reddit dataset statistics

	messages	users	% label 1	mean messages/user	median messages/user
Train	124638	7527	0.021	16.6	3
Test	15568	3440	0.021	4.5	2

can still be selected by random sampling. α_2 is our softmax temperature. α_3 is the proportion of users which are selected uniformly at random. In our experiments we used $\alpha_1 = 0.75, \alpha_2 = 0.01, \alpha_3 = 0.1$. We chose α_2 to ensure that the softmax did not produce $p_k = 0$ from underflow errors, and α_1, α_3 were both chosen based on initial experiments on Sticker Intent dataset. The underlying model trained with Federated Learning used a 64 dimensional character level embedding, a 32 dimensional BLSTM, and an MLP with one 64 dimensional hidden layer. The number of users in each Federated round was 200, and on each user 2 passes of SGD was performed with a batch size of 128. The learning rates for both local SGD and Federated ADAM were tuned separately for Random Sampling and AFL and the optimal learning rates were used for each.

Figure 2 shows the AUC after each Epoch under uniform random selection of users, and with AFL selection, showing mean and standard errors from 10 repetitions on test data. AFL trains models of the same performance using 20-70% fewer Epochs (where one Epoch is enough training rounds to train on each client once in expectation under random sampling).

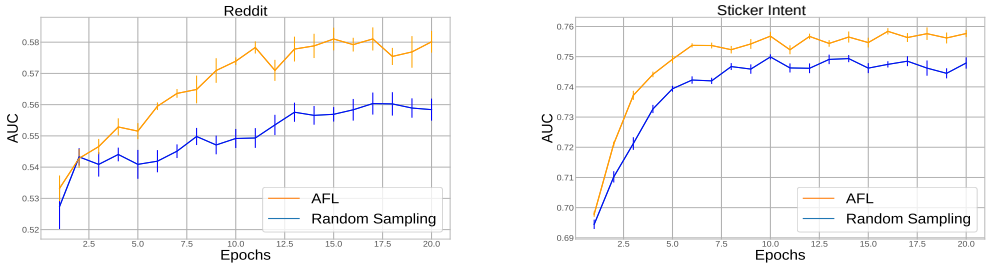


Figure 2: Comparison of AUC increase on Reddit and Sticker Intent datasets

One difference between AFL and server-side resampling techniques is that AFL selects data points by user, whereas server-side resampling can select arbitrary subsets. To explore the significance of this restriction we compared the gains from oversampling of label 1 data [7] and server-side learning against AFL using the value function $v_k = \sum \mathbf{1}_{y_{i,k}=1}$ and Federated training, using the Reddit dataset. The level of resampling and learning rates were tuned for server training, as were the temperature α_2 and the learning rates for Federated training, and all other tuning parameters were kept the same. Our results suggest that there is significant loss from selecting users, as the difference between Random Sampling and Active Sampling is much larger for server-side learning.

	Random Sampling	Active Sampling
Server selection of data points	0.559	0.615
Federated selection of clients	0.552	0.578

6 Conclusion and Further directions

In this paper we proposed Active Federated Learning (AFL), the first user cohort selection technique for FL which actively adapts to the state of the model and the data on each client. This adaptation allows us to train models with 20-70% fewer iterations for the same performance. Giving formal privacy guarantees is vital future work, but there are many other interesting extensions as well. These experiments were done under simplifying conditions which do not take into account many problems Federated Learning faces in practice, and which AFL may be able to help alleviate. For example clients may have different rates of availability for training. This availability may be correlated with the data on the client, resulting in bias in our model if not corrected. AFL which also takes reliability into account may be used to reduce this bias by increasing the rate at which we try to train

on unreliable users. Another challenge is that clients are constantly gathering (and potentially forgetting) data, and in many cases the distribution may be non-stationary. Maintaining the benefits of AFL may require a principled way of ensuring no user goes too long without having their valuation refreshed. Finally our experiments and analyses focused on the classification setting, but the loss value function can be used for any supervised problem, and understanding AFL with more complex models would be an interesting research direction.

References

[1] Bagdasaryan, E., Veit, A., Hua, Y., Estrin, D., and Shmatikov, V. (2018). How to backdoor federated learning. *arXiv preprint arXiv:1807.00459*.

[2] Baumgartner, J. (2019). Reddit Comments Dumps. `https://files.pushshift.io/reddit/comments/`. Accessed: 2019-09-03.

[3] Blaschzyk, I., Steinwart, I., et al. (2018). Improved classification rates under refined margin conditions. *Electronic Journal of Statistics*, 12(1):793–823.

[4] Caldas, S., Konečny, J., McMahan, H. B., and Talwalkar, A. (2018). Expanding the reach of federated learning by reducing client resource requirements. *arXiv preprint arXiv:1812.07210*.

[5] Dwork, C., Roth, A., et al. (2014). The algorithmic foundations of differential privacy. *Foundations and Trends® in Theoretical Computer Science*, 9(3–4):211–407.

[6] Hartmann, F. (2018). Federated learning.

[7] He, H. and Garcia, E. A. (2008). Learning from imbalanced data. *IEEE Transactions on Knowledge & Data Engineering*, (9):1263–1284.

[8] Konečnỳ, J., McMahan, H. B., Yu, F. X., Richtárik, P., Suresh, A. T., and Bacon, D. (2016). Federated learning: Strategies for improving communication efficiency. *arXiv preprint arXiv:1610.05492*.

[9] Leroy, D., Coucke, A., Lavril, T., Gisselbrecht, T., and Dureau, J. (2019). Federated learning for keyword spotting. In *ICASSP 2019-2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 6341–6345. IEEE.

[10] Li, T., Sahu, A. K., Talwalkar, A., and Smith, V. (2019). Federated learning: Challenges, methods, and future directions. *arXiv preprint arXiv:1908.07873*.

[11] McMahan, H. B., Moore, E., Ramage, D., Hampson, S., et al. (2016). Communication-efficient learning of deep networks from decentralized data. *arXiv preprint arXiv:1602.05629*.

[12] Nishio, T. and Yonetani, R. (2019). Client selection for federated learning with heterogeneous resources in mobile edge. In *ICC 2019-2019 IEEE International Conference on Communications (ICC)*, pages 1–7. IEEE.

[13] Settles, B. (2009). Active learning literature survey. Technical report, University of Wisconsin-Madison Department of Computer Sciences.

[14] Yang, Q., Liu, Y., Chen, T., and Tong, Y. (2019). Federated machine learning: Concept and applications. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 10(2):12.